

Decomposition Methods for Two-Stage Stochastic Programming

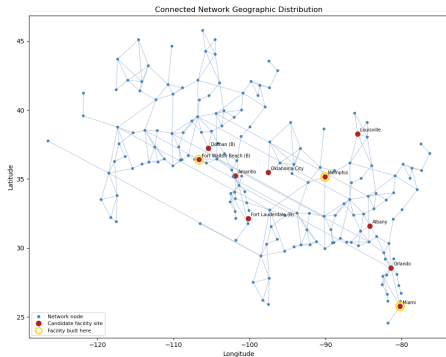
Rudra Powale¹ Jimmy Lin²

¹University of Southampton

²STOR-i Centre for Doctoral Training

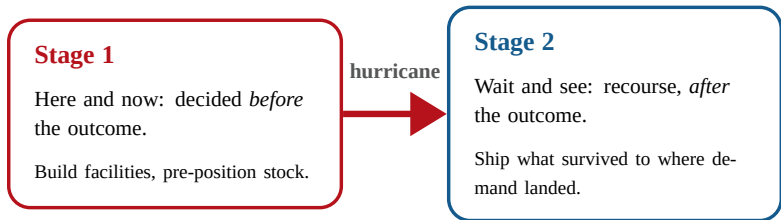
The Case Study

The Network and the Scenarios



Quantity	Count
Nodes	180
Facility sizes	3
Commodities	3
Scenarios	500

Two-Stage Model



Building the Model

Stage 1 Decision Variables

- y_{il} – binary variable to decide if we should build a facility at location i
- r_i^k – how much of commodity k to stock there

Stage 2 Decision Variables

- x_{ij}^{ks} – flow of commodity k on link (i, j)
- z_i^{ks} – leftover stock
- w_i^{ks} – unmet demand

The Two-Stage Formulation

Stage 1: $\min_x c^\top x + \mathbb{E}_\xi[Q(x, \xi)] \quad \text{s.t. } Ax \geq b, x \geq 0$

Stage 2: $Q(x, \xi) = \min_y q(\xi)^\top y \quad \text{s.t. } W(\xi)y \geq h(\xi) - T(\xi)x, y \geq 0$

With finitely many scenarios this collapses into one optimisation problem, which is the **deterministic equivalent**.

Objective – What Each Term Means

$$\min \sum_{i \in I} \sum_{l \in L} F_{il} y_{il} \quad + \quad \sum_{k \in K} \sum_{i \in I} q^k r_i^k \quad +$$

 **Facility Setup Costs**  **Acquisition Costs**

$$\sum_{s \in S} P_s \left[\sum_{(i,j) \in A} \sum_{k \in K} c_{ij}^{ks} x_{ij}^{ks} \quad + \quad \sum_{i \in I} \sum_{k \in K} h^k z_i^{ks} \quad + \quad \sum_{i \in I} \sum_{k \in K} p^k w_i^{ks} \right]$$

 **Expected Scenario Costs**  **Transportation Costs**  **Holding Costs**  **Penalty Costs**

- Supply balances at every node:

$$\sum_{j \neq i} x_{ji}^{ks} + \rho_i^{ks} r_i^k - z_i^{ks} = \sum_{j \neq i} x_{ij}^{ks} + v_i^{ks} - w_i^{ks}$$

Constraints – What Has to Hold True

- Supply balances at every node:

$$\sum_{j \neq i} x_{ji}^{ks} + \rho_i^{ks} r_i^k - z_i^{ks} = \sum_{j \neq i} x_{ij}^{ks} + v_i^{ks} - w_i^{ks}$$

- Links have limited capacity:

$$\sum_{k \in K} u^k x_{ij}^{ks} \leq U_{ij}^s$$

- Facilities have limited space:

$$\sum_{k \in K} b^k r_i^k \leq \sum_{l \in L} M_l y_{il}$$

- Only one size per site:

$$\sum_{l \in L} y_{il} \leq 1 \quad \forall i \in I$$

- Non-negativity constraints:

$$x_{ij}^{ks}, z_i^{ks}, w_i^{ks}, r_i^k \geq 0, \quad y_{il} \in \{0, 1\}$$

Solving This as One Big Problem

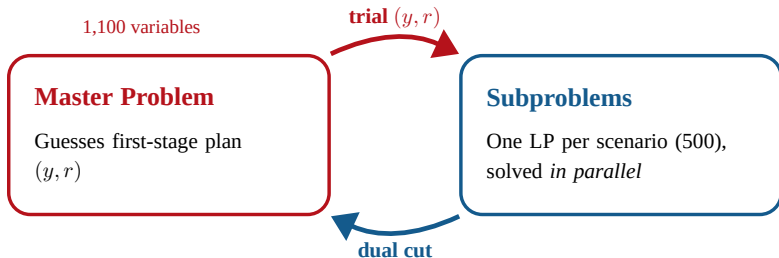
Putting every constraint above together across all 500 scenarios gives one large **deterministic equivalent** to solve directly.

Variables	1,509,084
Constraints	270,360

This is what motivates **decomposition** – solving it all at once doesn't scale.

Solving via Benders Decomposition

Benders Decomposition



Optimality Cut: feasible subproblem $\rightarrow$ dual prices give a tight lower bound on the expected recourse cost, added to the master as

$$\theta \geq \alpha_\tau + \beta_\tau^\top x$$

Feasibility Cut: infeasible subproblem $\rightarrow$ dual rays show that trial solution is impossible, added to the master as

$$(\sigma^*)^\top (h_s - T_s x) \leq 0$$

Cuts, Illustrated

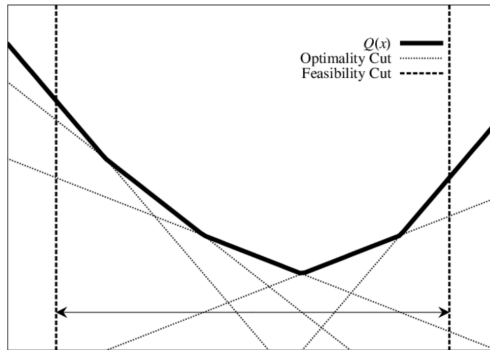


Figure. Illustration of Benders decomposition ([shiina2016multistage](#))

Two Benders Variants

Single-cut: averages all scenarios into **one** combined cut per iteration.

Multi-cut: generates **one cut per scenario**, adding all of them to the master every iteration.

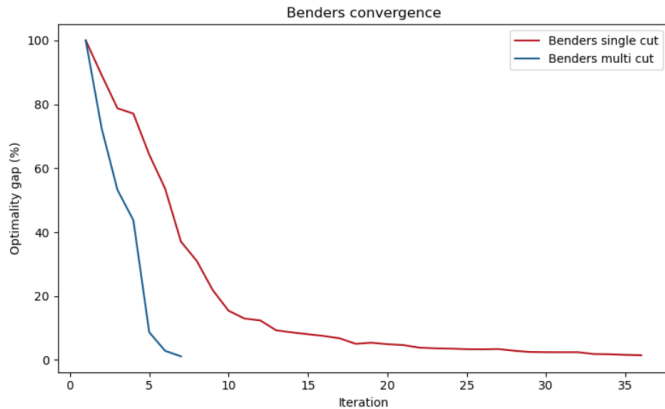
	Single-cut	Multi-cut
Cuts added per iteration	1	500
Information per cut	Averaged	Scenario-specific

Computational Results

Results – Method Comparison

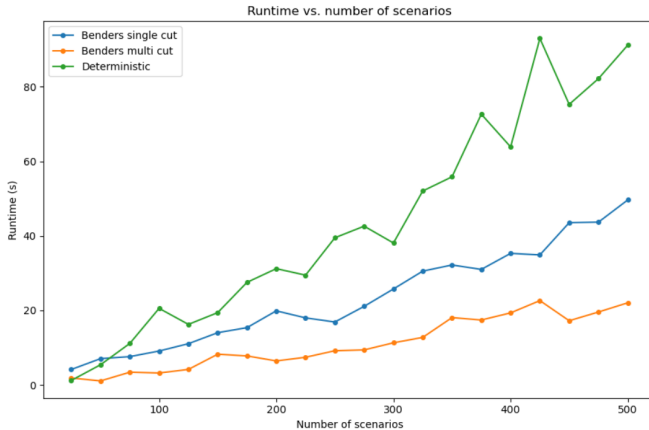
Method	Iterations	Time (s)	Gap (%)	Objective
Benders single cut	36	41.9	1.478	73,206,665
Benders multi cut	7	23.8	1.131	73,345,309
Deterministic	1	92.4	1.429	73,787,984

Results – Convergence of the Bounds



Runtime Scaling

Runtime Scaling



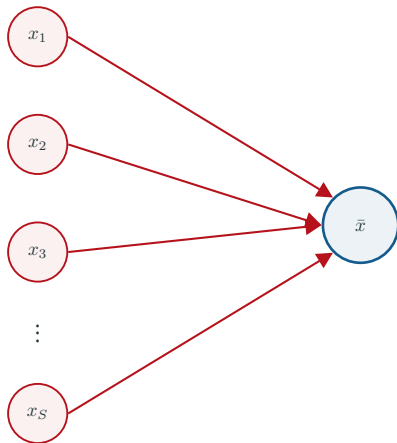
Lagrangian Relaxation

The Big Idea

each scenario gets its own copy

non-anticipativity:

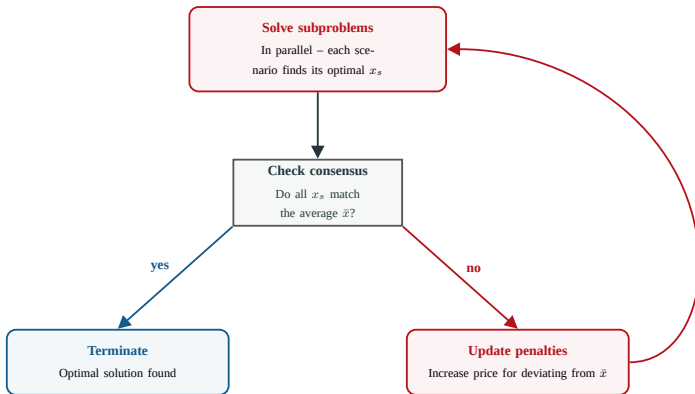
$$x_s = \bar{x} \quad \forall s$$



all copies
must agree

Instead of enforcing agreement directly, we *price* it – what's left splits into S small, independent LPs, solved in parallel.

The Lagrangian Algorithm



Conclusion

- The deterministic equivalent is computationally expensive for a large number of scenarios.
- **Benders decomposition** splits the problem into a small master (1,100 variables) and 500 independent scenario subproblems, solved in parallel.
- Multi-cut Benders converged fastest – 7 iterations, 23.8s, vs. 36 iterations and 41.9s for single-cut.
- Both Benders variants comfortably outperformed solving the deterministic equivalent directly.
- **Lagrangian relaxation** is another powerful decomposition method, but does not converge at the optimal solution for this problem.

Thank You

Any Questions?